

CS 342: Computer Vision and Pattern Recognition

Instructor: Br. Tamal Mj

Shanti Mantra

Lyrics in Sanskrit

Meaning in English

ॐ सह नाववतु।

Om, May we all be protected

सह नौ भुनक्तु।

May we all be nourished

सह वीर्यं करवावहै।

May we work together with great energy

तेजस्यि नावधीतमस्तु

May our intellect be sharpened (may our study be effective)

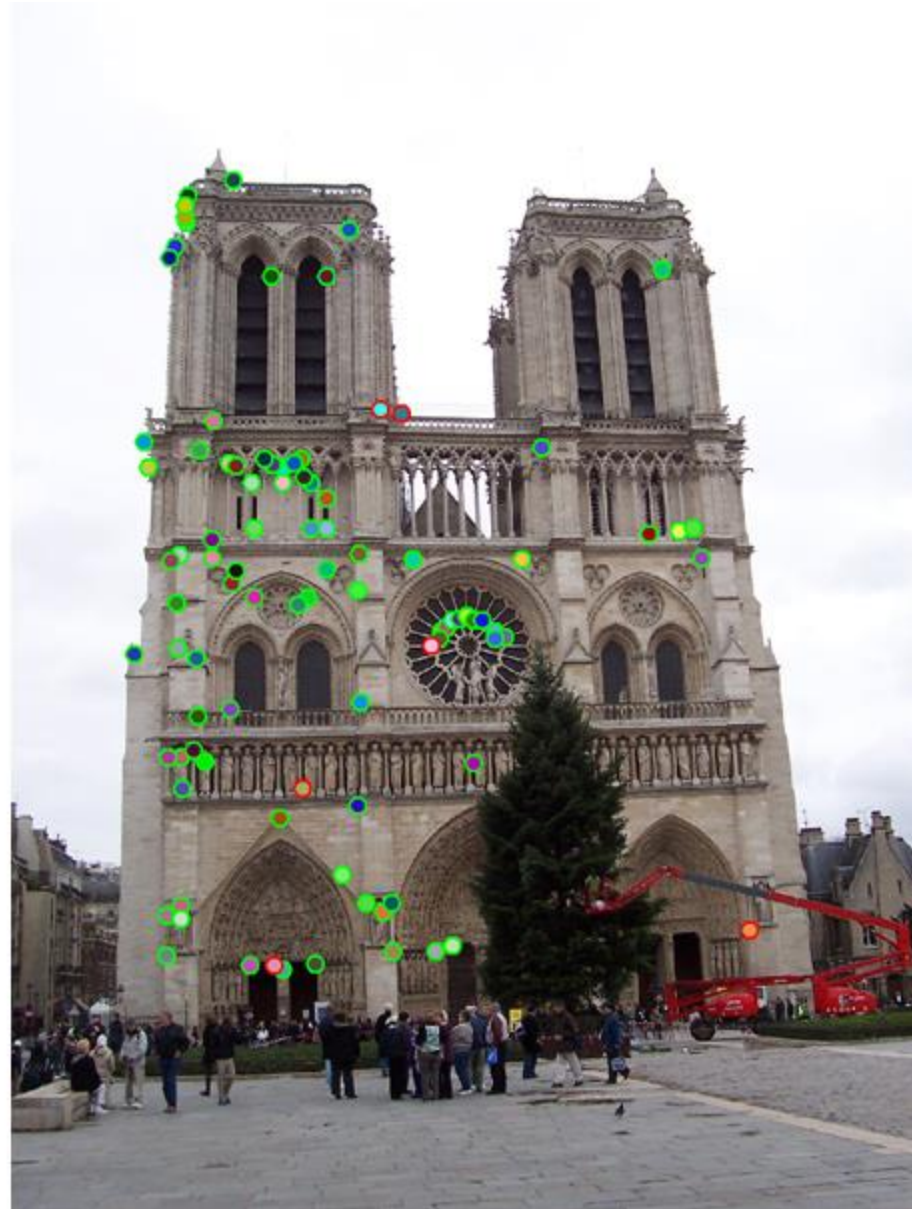
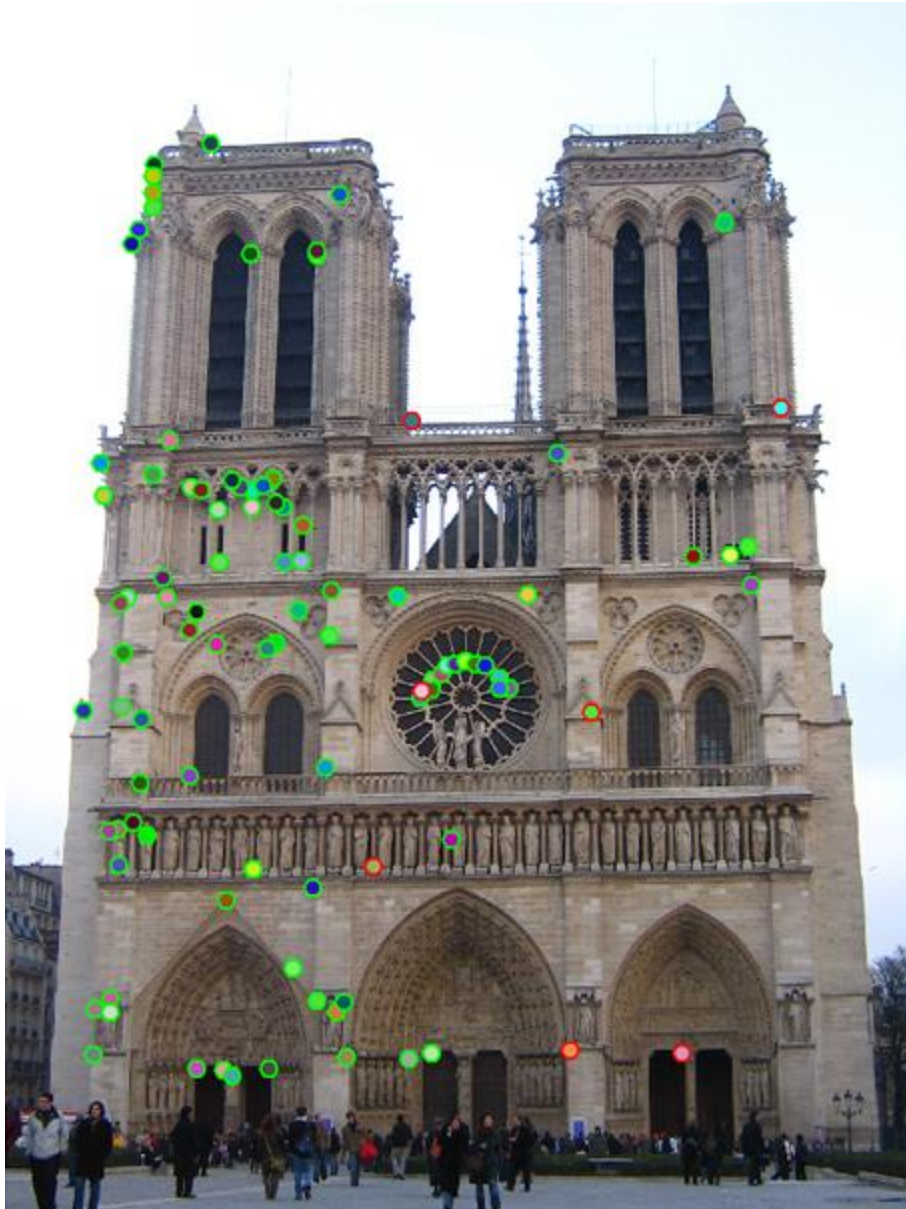
मा विद्विषावहै।

Let there be no Animosity amongst us

ॐ शान्तिः शान्तिः शान्तिः ॥

Om, peace (in me), peace (in nature), peace (in divine forces)

Can we refine this further?



Fitting and Alignment

Fitting: find the parameters of a model that best fit the data

Alignment: find the parameters of the transformation that best align matched points

Fitting and Alignment

- Design challenges
 - Design a suitable **goodness of fit** measure
 - Similarity should reflect application goals
 - Encode robustness to outliers and noise
 - Design an **optimization** method
 - Avoid local optima
 - Find best parameters quickly

Fitting and Alignment: Methods

- Global optimization / Search for parameters
 - Least squares fit
 - Other parameter search methods
- Hypothesize and test
 - Generalized Hough transform
 - RANSAC

Fitting and Alignment: Methods

- Global optimization / Search for parameters
 - Least squares fit
 - Other parameter search methods
- Hypothesize and test
 - Generalized Hough transform
 - RANSAC

Simple example: Fitting a line

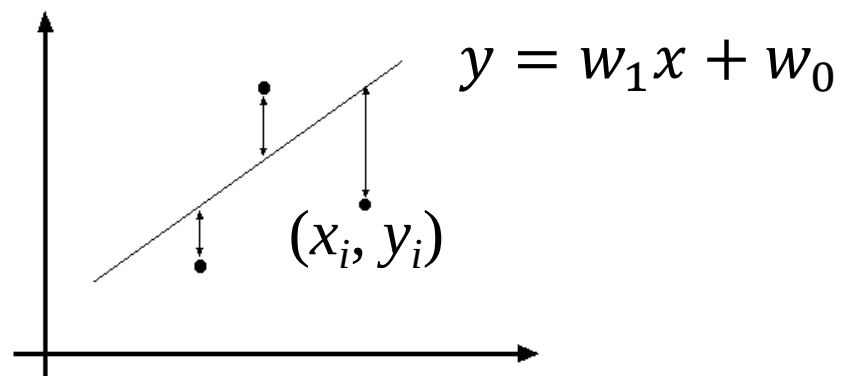
Least squares line fitting

- Data: $(x_1, y_1), \dots, (x_n, y_n)$

Line equation: $y = w_1x + w_0$

- Find (w_0, w_1) to minimize

$$E = \sum_{i=1}^n (y_i - w_1x_i - w_0)^2$$



Rewriting using **matrix notation**:

$$E = \|Xw - y\|^2$$

where:

$$X = \begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{bmatrix}, \quad w = \begin{bmatrix} w_1 \\ w_0 \end{bmatrix}, \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

By solving for w using the normal equation:

$$X^T X w = X^T y$$

$$w = (X^T X)^{-1} X^T y$$

In python:

```
W = np.linalg.lstsq(X, y)
```

Least squares (global) optimization

Good

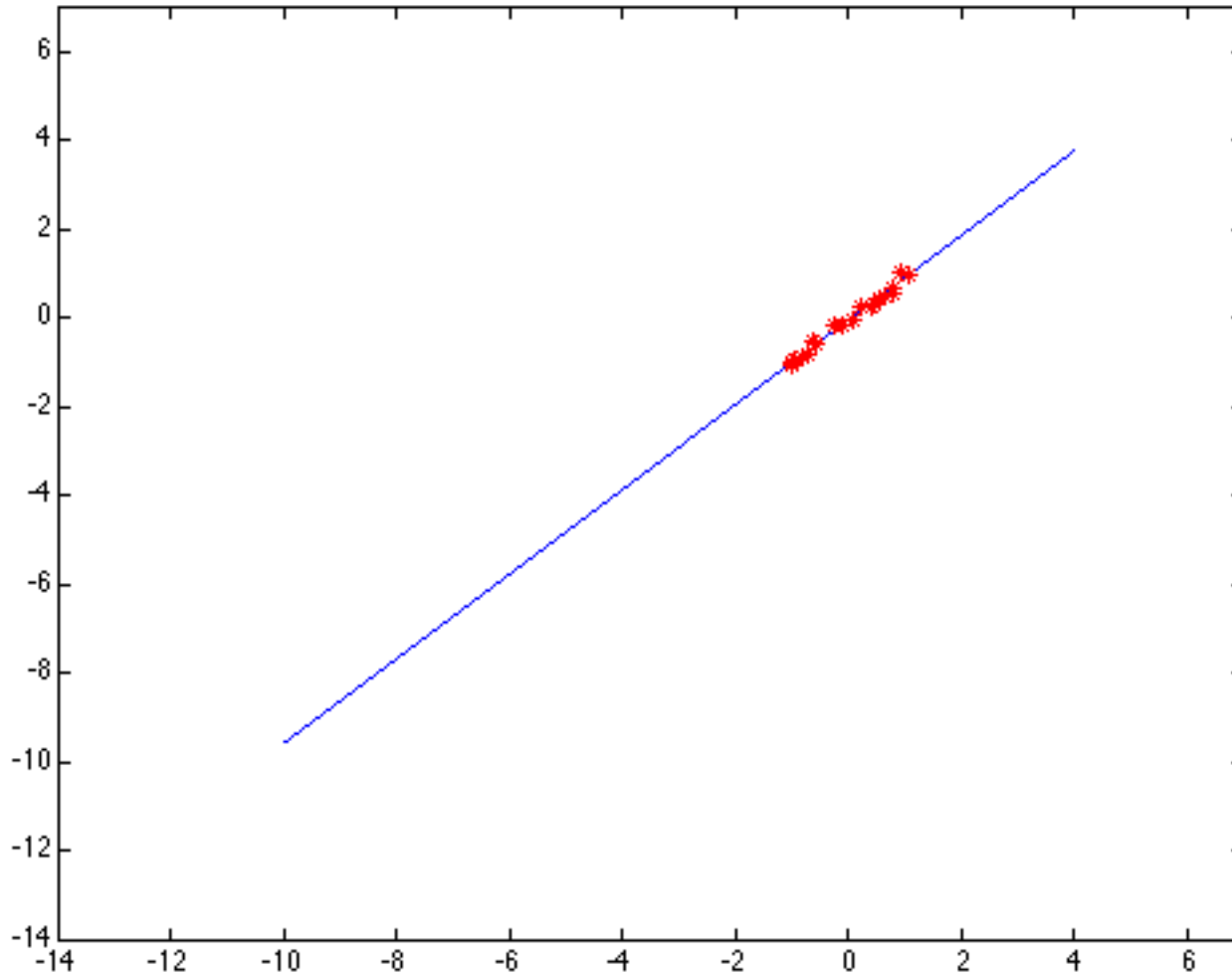
- Clearly specified objective
- Optimization is easy (has a **closed-form solution**)

Bad

- May not be what you want to optimize
 - assumes **Gaussian noise** in the data
- Sensitive to outliers
 - Bad matches, extra points
- Doesn't allow you to get multiple good fits
 - Detecting multiple objects, lines, etc.

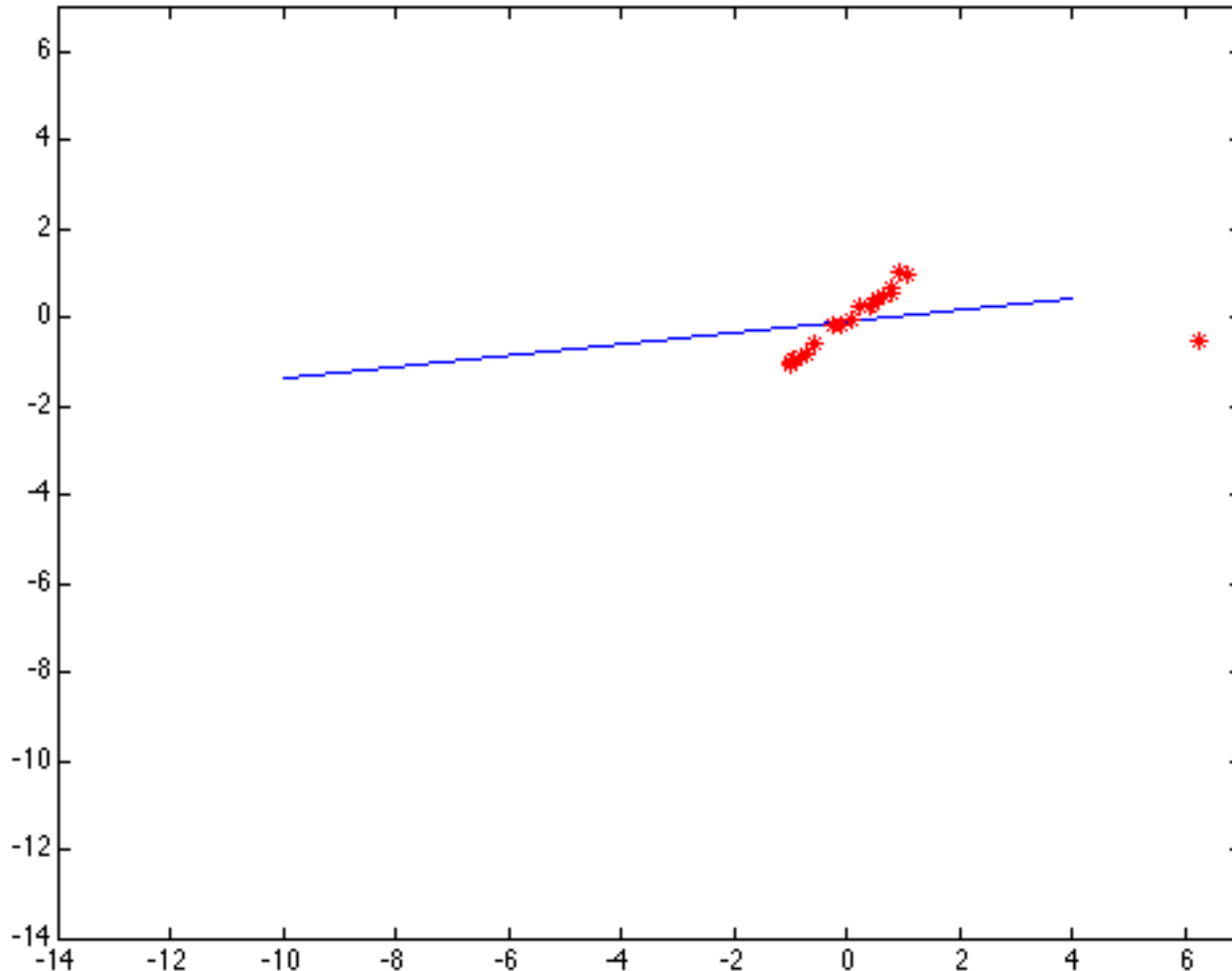
Least squares: Robustness to noise

- Least squares fit to the red points:



Least squares: Robustness to noise

- Least squares fit with an outlier:



Problem: squared error heavily penalizes outliers

Fitting and Alignment: Methods

- Global optimization / Search for parameters
 - Least squares fit
 - Other parameter search methods
- Hypothesize and test
 - Generalized Hough transform
 - RANSAC

Other ways to search for parameters (for when no closed form solution exists)

- Line search
 1. For each parameter, step through values and choose value that gives best fit
 2. Repeat (1) until no parameter changes
- Grid search
 1. Propose several sets of parameters, evenly sampled in the joint set
 2. Choose best (or top few) and sample joint parameters around the current best; repeat
- Gradient descent
 1. Provide initial position (e.g., random)
 2. Locally search for better parameters by following gradient

Fitting and Alignment: Methods

- Global optimization / Search for parameters
 - Least squares fit
 - Robust least squares
 - Other parameter search methods
- Hypothesize and test
 - Generalized Hough transform
 - RANSAC

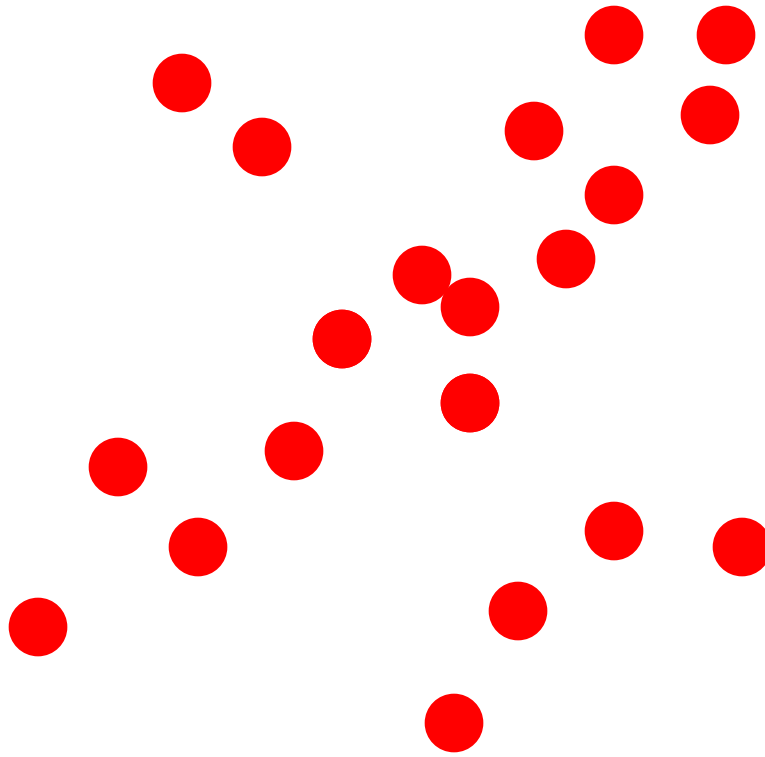
Fitting and Alignment: Methods

- Global optimization / Search for parameters
 - Least squares fit
 - Robust least squares
 - Other parameter search methods)
- Hypothesize and test
 - Generalized Hough transform
 - RANSAC

RANSAC

(**R**ANdom **S**Ample **C**onsensus) :

Fischler & Bolles in '81.



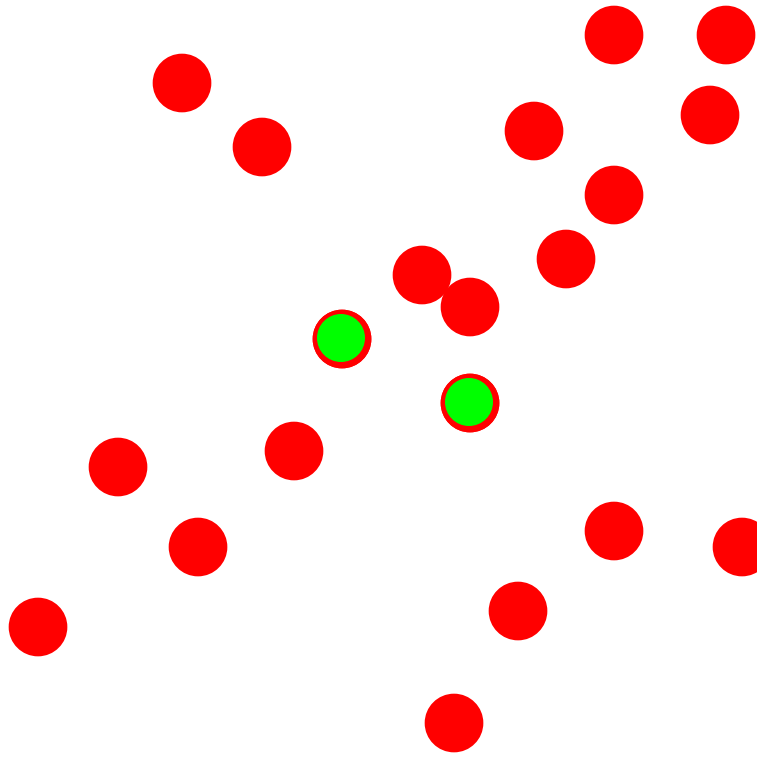
Algorithm:

1. **Sample** (randomly) the number of points required to fit the model
2. **Solve** for model parameters using samples
3. **Score** by the fraction of inliers within a preset threshold of the model

Repeat 1-3 until the best model is found with high confidence

RANSAC

Line fitting example



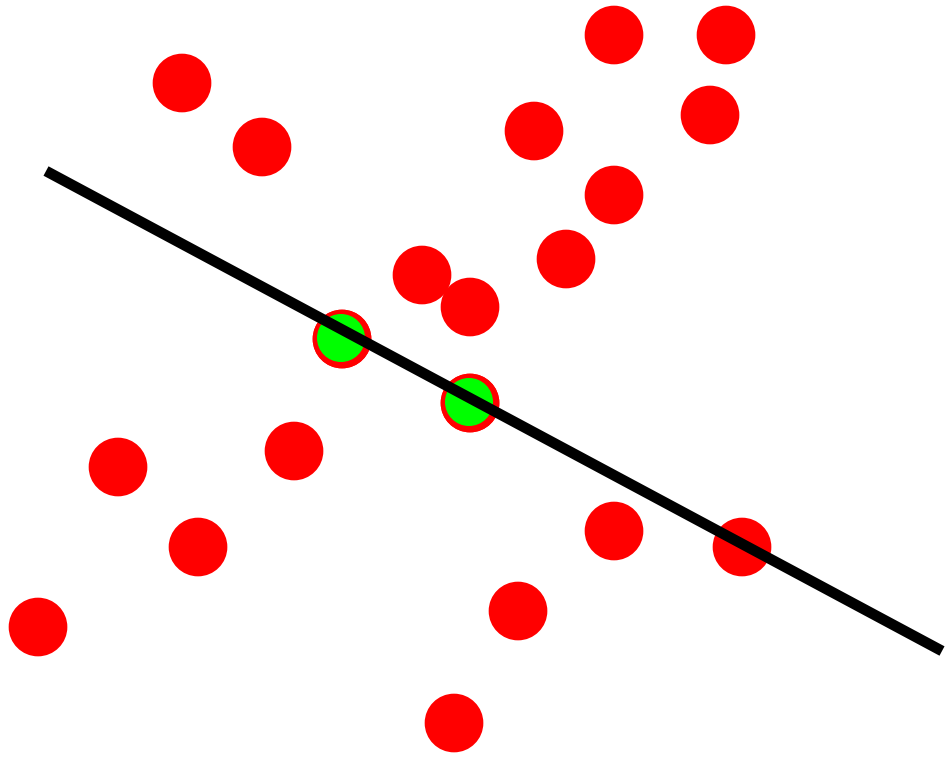
Algorithm:

1. **Sample** (randomly) the number of points required to fit the model ($\# = 2$)
2. **Solve** for model parameters using samples
3. **Score** by the fraction of inliers within a preset threshold of the model

Repeat 1-3 until the best model is found with high confidence

RANSAC

Line fitting example



Algorithm:

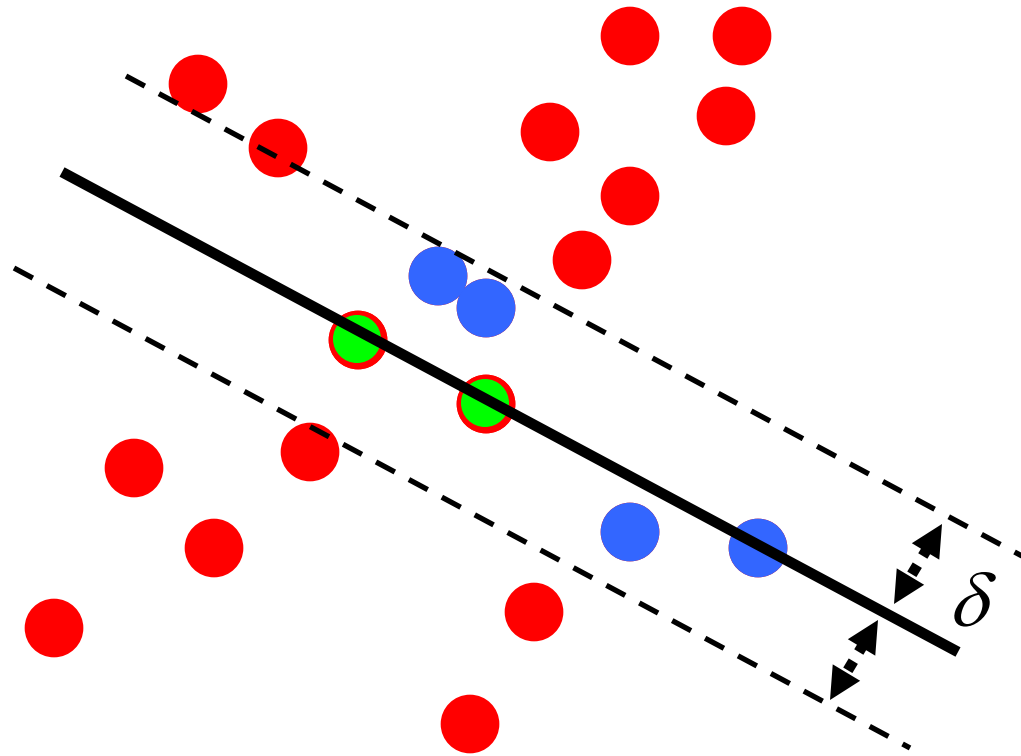
1. **Sample** (randomly) the number of points required to fit the model ($\# = 2$)
2. **Solve** for model parameters using samples
3. **Score** by the fraction of inliers within a preset threshold of the model

Repeat 1-3 until the best model is found with high confidence

RANSAC

Line fitting example

$$N_I = 6$$

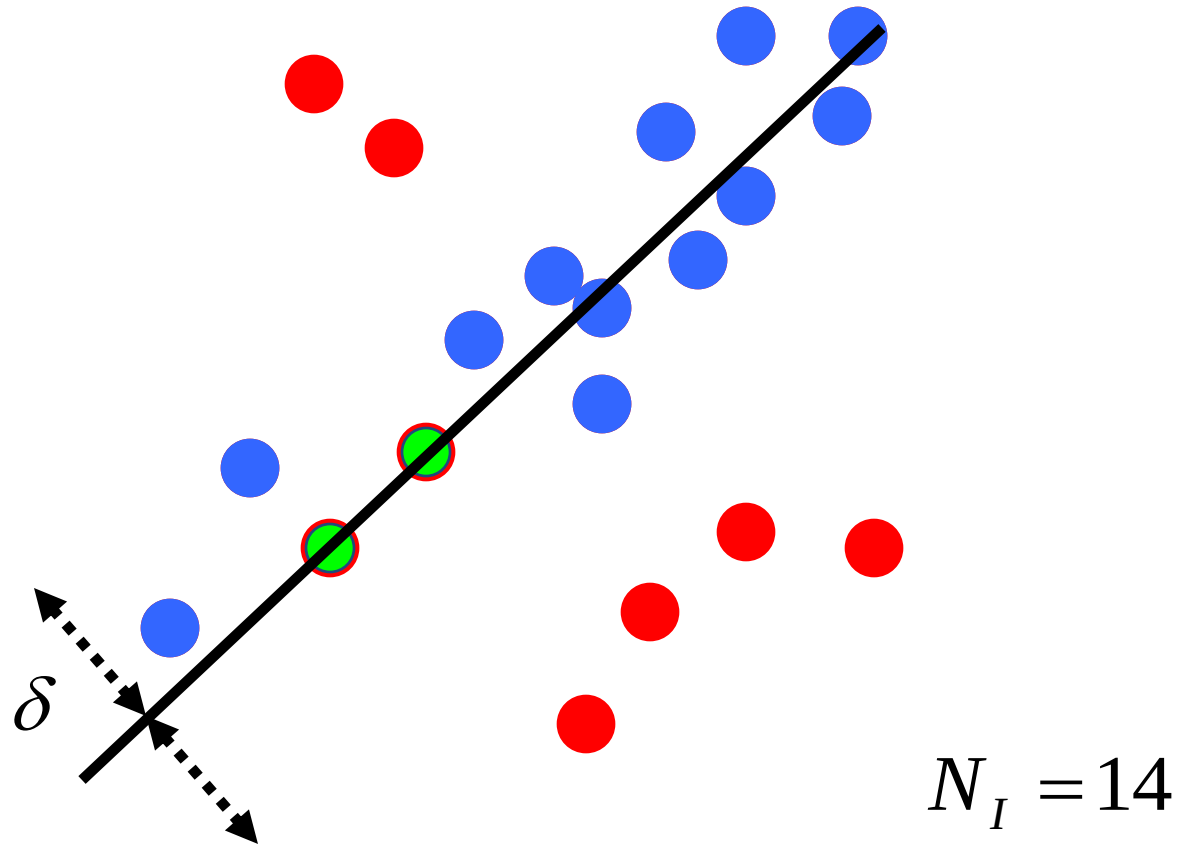


Algorithm:

1. **Sample** (randomly) the number of points required to fit the model ($\# = 2$)
2. **Solve** for model parameters using samples
3. **Score** by the fraction of inliers within a preset threshold of the model

Repeat 1-3 until the best model is found with high confidence

RANSAC



Algorithm:

1. **Sample** (randomly) the number of points required to fit the model ($\# = 2$)
2. **Solve** for model parameters using samples
3. **Score** by the fraction of inliers within a preset threshold of the model

Repeat 1-3 until the best model is found with high confidence

RANSAC Algorithm

RANSAC operates in an iterative manner:

1. **Randomly select s points** (minimal subset needed to fit the model).
2. **Fit a model** using only these s points.
3. **Classify inliers:** Check how many other points are close to the model (within a threshold δ).
4. **Repeat for N iterations**, keeping the model with the highest number of inliers.
5. **Recompute the model** using all inliers from the best sample.

Determining N (Number of Iterations)

The probability that a **single** randomly chosen sample of size s contains **only inliers** is:

$$(1 - e)^s$$

s = # of points to compute
 p = probability of success
 e = proportion of outliers

The probability that a **single** randomly chosen sample of size s contains **at least one outliers** is:

$$1 - (1 - e)^s$$

The probability that all N randomly chosen sample of size s contains **at least one outliers** is:

$$(1 - (1 - e)^s)^N$$

We want: $(1 - (1 - e)^s)^N < 1 - p$

We require that the probability of **all bad samples** is less than $1 - p$

Determining N (Number of Iterations)

- We require that the probability of **all bad samples** is **less than** $1 - p$

$$(1 - (1 - e)^s)^N < 1 - p$$

- This ensures that **at least one** sample is fully inlier-free with probability **greater than** p
- Rearranging $N \log(1 - (1 - e)^s) < \log(1 - p)$

Solving for N (the number of iterations)

$$N > \frac{\log(1 - p)}{\log(1 - (1 - e)^s)}$$

How to choose parameters?

- Number of samples N
 - Choose N so that, with probability p , at least one random sample is free from outliers (e.g. $p=0.99$) (outlier ratio: e)
- Number of sampled points s
 - Minimum number needed to fit the model
- Distance threshold δ
 - Choose δ so that a good point with noise is likely (e.g., prob=0.95) within threshold
 - Zero-mean Gaussian noise with std. dev. σ : $t^2=3.84\sigma^2$

$$N > \frac{\log(1 - p)}{\log(1 - (1 - e)^s)}$$

proportion of outliers e							
s	5%	10%	20%	25%	30%	40%	50%
2	2	3	5	6	7	11	17
3	3	4	7	9	11	19	35
4	3	5	9	13	17	34	72
5	4	6	12	17	26	57	146
6	4	7	16	24	37	97	293
7	4	8	20	33	54	163	588
8	5	9	26	44	78	272	1177

For $p = 0.99$

RANSAC conclusions

Good

- Robust to outliers
- Applicable for larger number of model parameters than Hough transform
- Optimization parameters are easier to choose than Hough transform

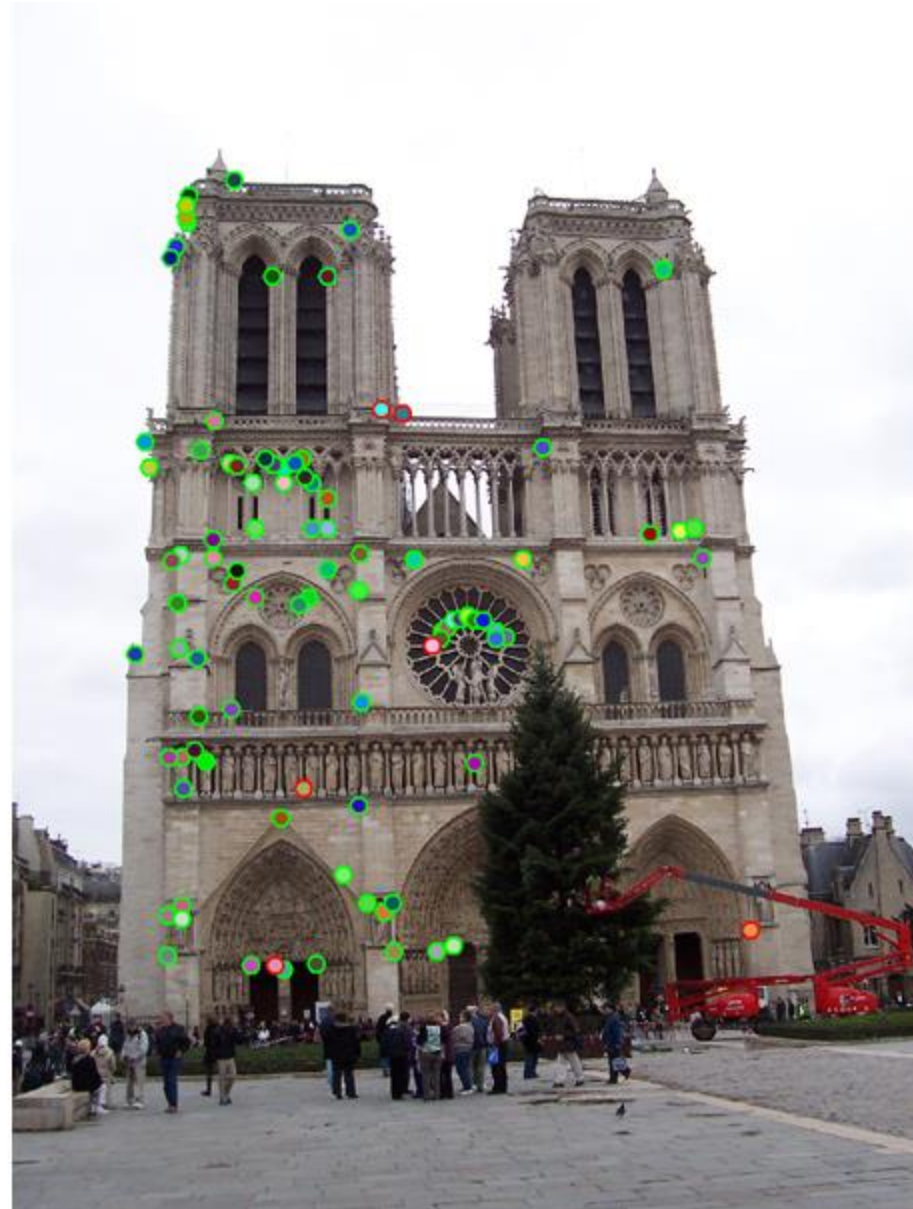
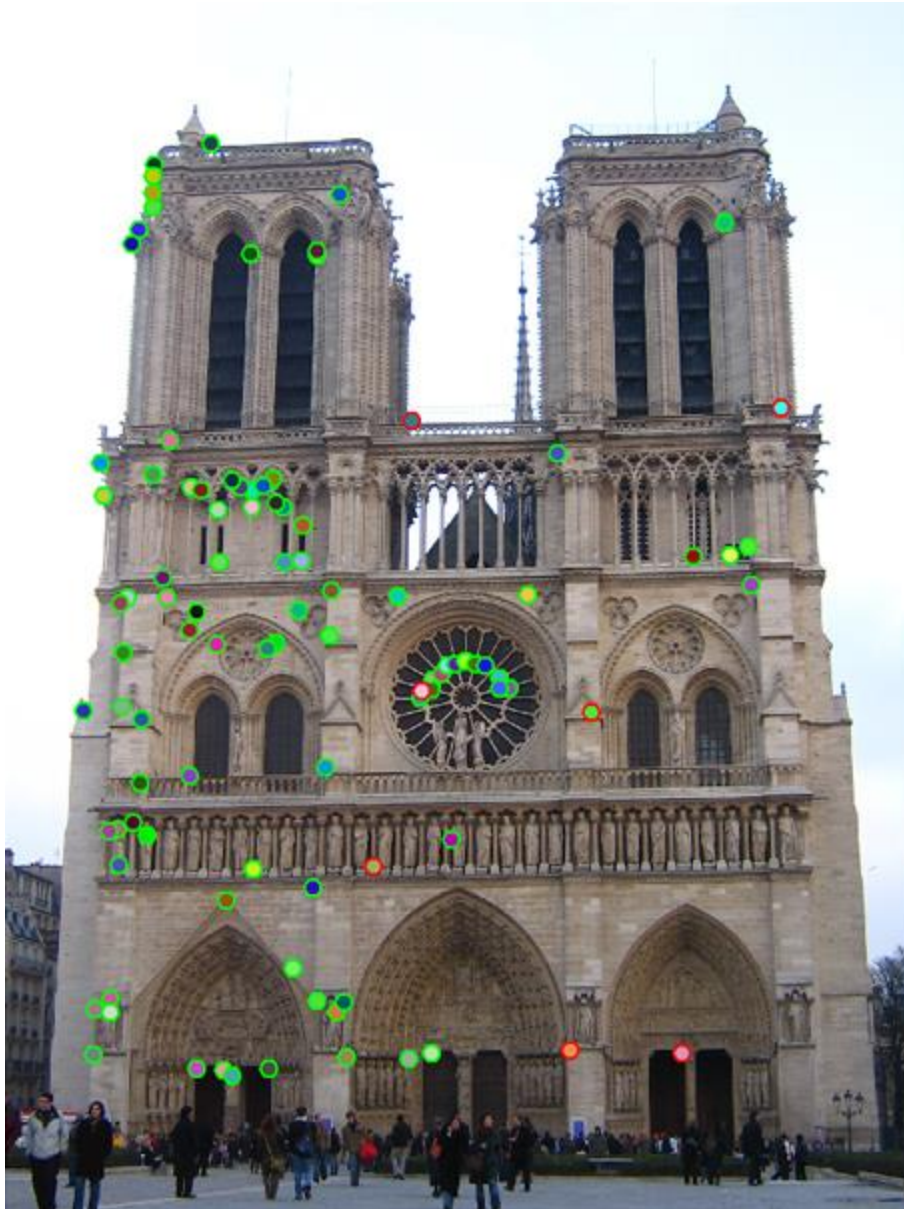
Bad

- Computational time grows quickly with fraction of outliers and number of parameters
- Not good for getting multiple fits

Common applications

- Computing a homography (e.g., image stitching)
- Estimating fundamental matrix (relating two views)

How do we fit the best alignment?



Alignment

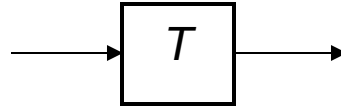
Alignment: find parameters of model that maps one set of points to another

- Typically want to solve for a global transformation that accounts for **most** true correspondences
- Difficulties
 - Noise (**shifts feature locations typically by 1-3 pixels**)
 - Outliers (often 50%)

Parametric (global) warping



$$\mathbf{p} = (x, y)$$



$$\mathbf{p}' = (x', y')$$

Transformation T is a coordinate-changing machine:

$$p' = T(p)$$

What does it mean that T is global?

- Is the same for any point p
- can be described by just a few numbers (parameters)

For linear transformations, we can represent T as a matrix

$$p' = \mathbf{T}p$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \mathbf{T} \begin{bmatrix} x \\ y \end{bmatrix}$$

Common transformations



original

Transformed



translation



rotation



aspect



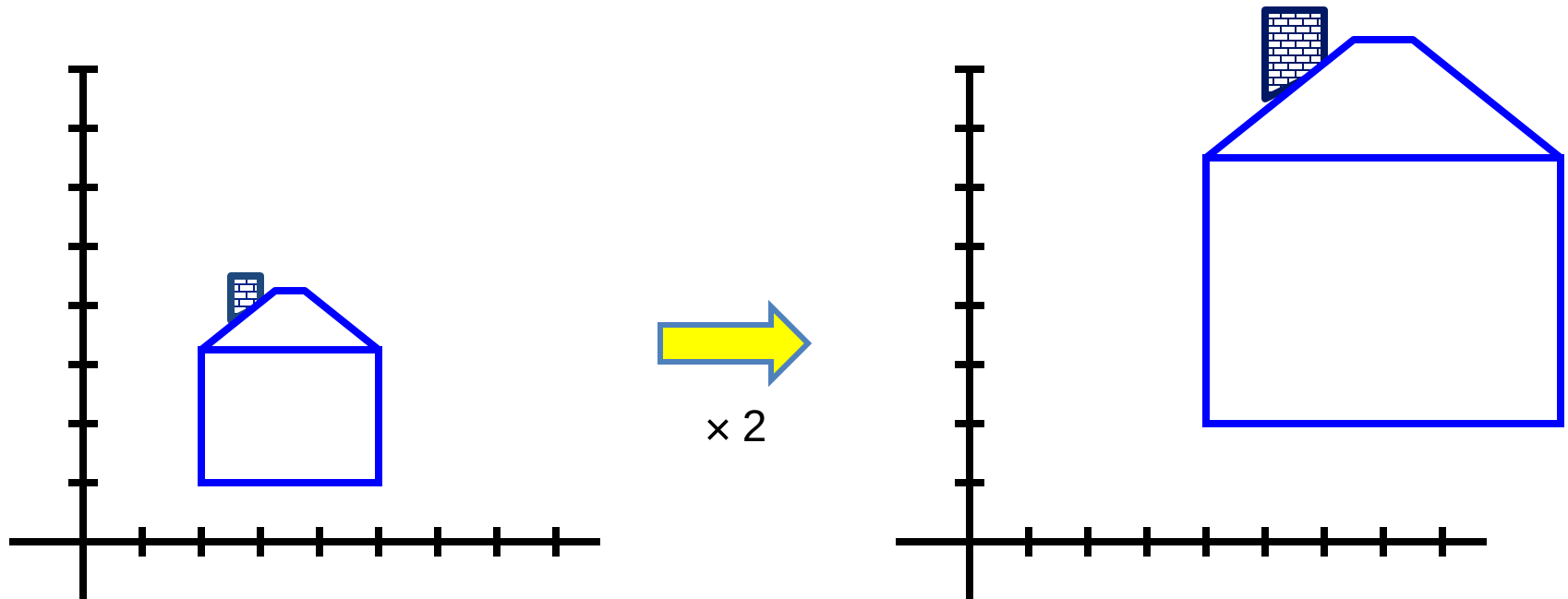
affine



perspective

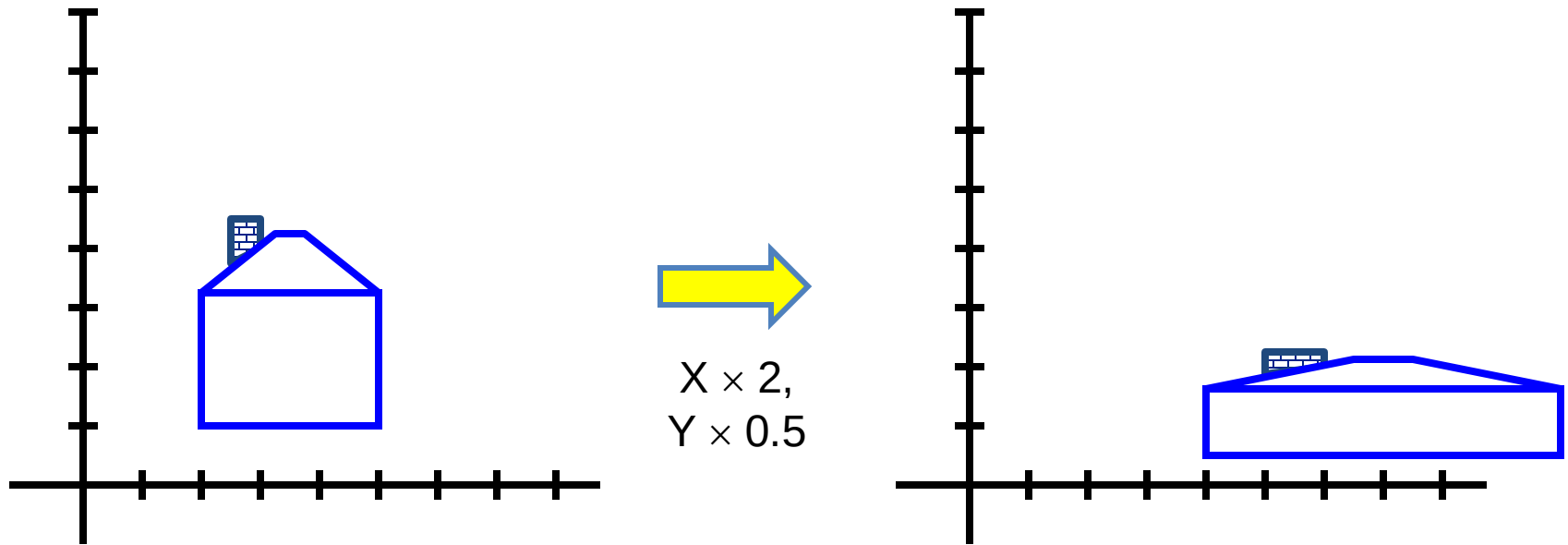
Scaling

- *Scaling* a coordinate means multiplying each of its components by a scalar
- *Uniform scaling* means this scalar is the same for all components:



Scaling

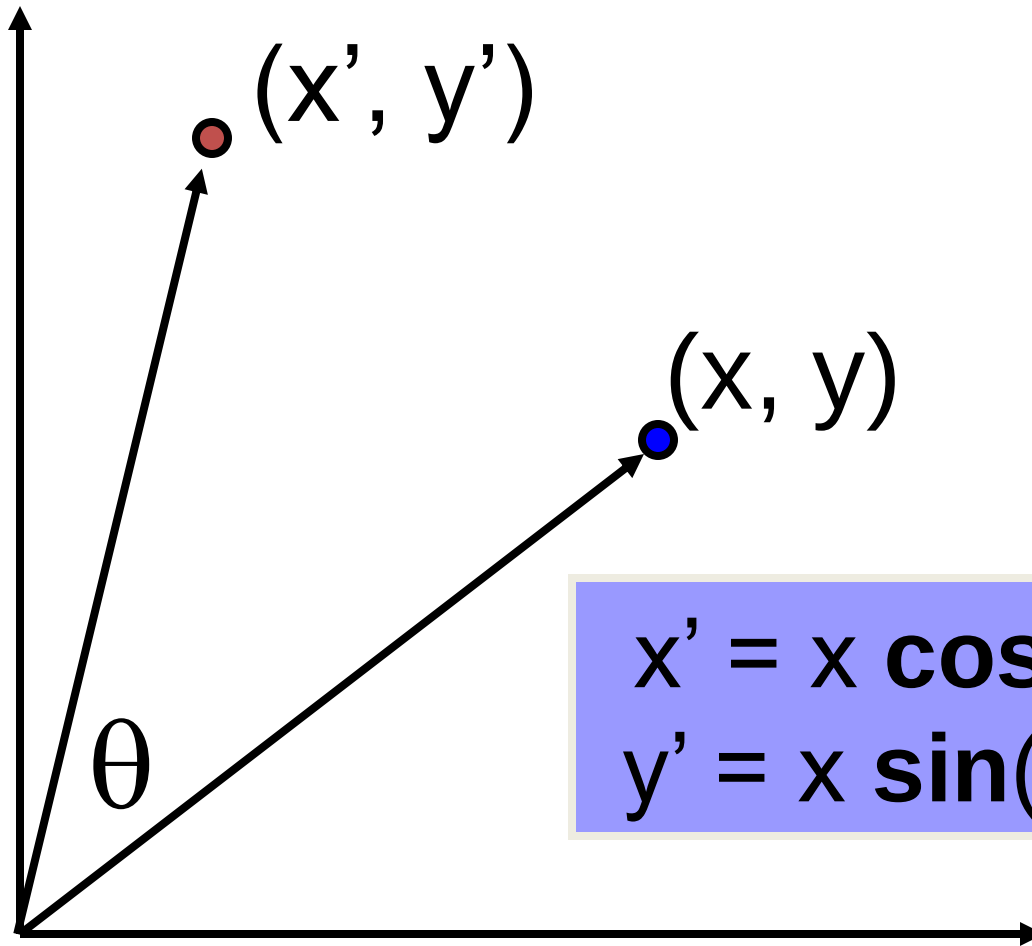
- *Non-uniform scaling*: different scalars per component:



Scaling

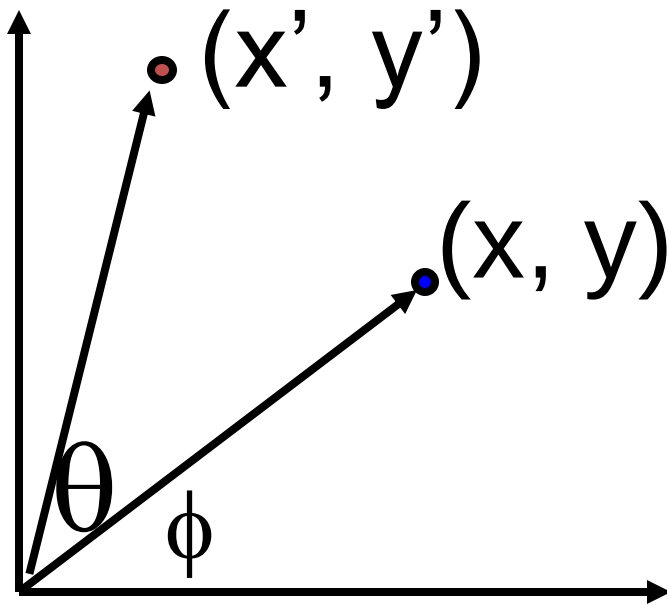
- Scaling operation: $x' = ax$
 $y' = by$
- Or, in matrix form:
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \underbrace{\begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix}}_{\text{scaling matrix } S} \begin{bmatrix} x \\ y \end{bmatrix}$$

2-D Rotation



$$\begin{aligned}x' &= x \cos(\theta) - y \sin(\theta) \\y' &= x \sin(\theta) + y \cos(\theta)\end{aligned}$$

2-D Rotation



Polar coordinates...

$$x = r \cos(\phi)$$

$$y = r \sin(\phi)$$

$$x' = r \cos(\phi + \theta)$$

$$y' = r \sin(\phi + \theta)$$

Trig Identity...

$$x' = r \cos(\phi) \cos(\theta) - r \sin(\phi) \sin(\theta)$$

$$y' = r \sin(\phi) \cos(\theta) + r \cos(\phi) \sin(\theta)$$

Substitute...

$$x' = x \cos(\theta) - y \sin(\theta)$$

$$y' = x \sin(\theta) + y \cos(\theta)$$

2-D Rotation

This is easy to capture in matrix form:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \underbrace{\begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}}_{\mathbf{R}} \begin{bmatrix} x \\ y \end{bmatrix}$$

Even though $\sin(\theta)$ and $\cos(\theta)$ are nonlinear functions of θ ,

- ***x' is a linear combination of x and y***
- ***y' is a linear combination of x and y***

What is the inverse transformation?

- Rotation by $-\theta$
- For rotation matrices **$\mathbf{R}^{-1} = \mathbf{R}^T$**

Basic 2D transformations

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Scale

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & \alpha_x \\ \alpha_y & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Shear

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \Theta & -\sin \Theta \\ \sin \Theta & \cos \Theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Rotate

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Translate

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Affine

Affine is any combination of translation, scale, rotation, shear

Affine Transformations

Affine transformations are combinations of

- Linear transformations, and
- Translations

Properties of affine transformations:

- Lines map to lines
- Parallel lines remain parallel
- Ratios are preserved
- Closed under composition

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

or

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Projective Transformations (also known as **homography**)

Projective transformations are combos of

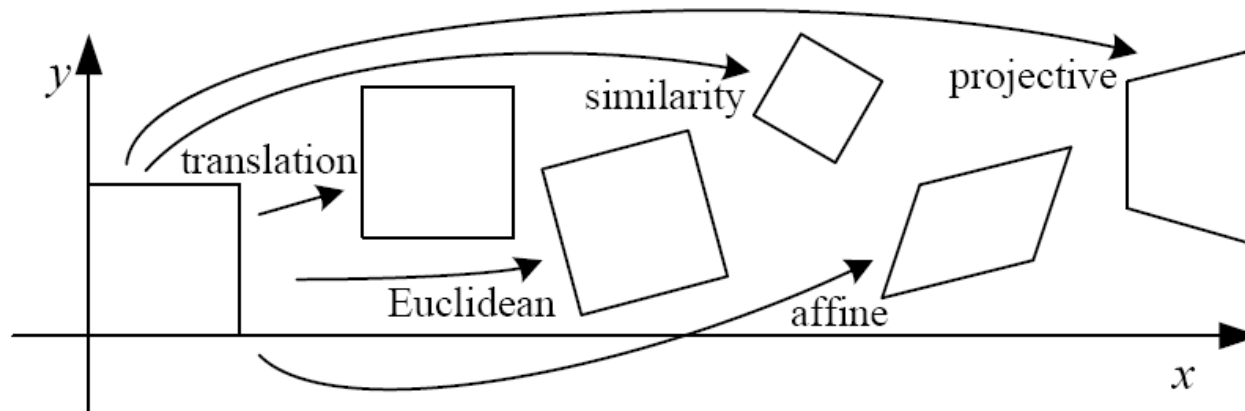
- Affine transformations, and
- Projective warps

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Properties of projective transformations:

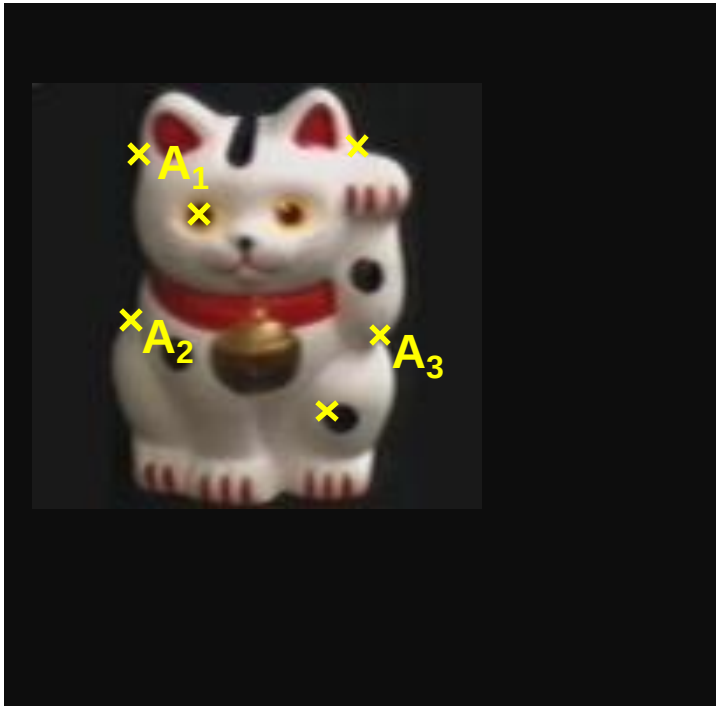
- Lines map to lines
- Parallel lines do not necessarily remain parallel
- Ratios are not preserved

2D image transformations (reference table)



Name	Matrix	# D.O.F.	Preserves:	Icon
translation	$\begin{bmatrix} I & t \end{bmatrix}_{2 \times 3}$	2	orientation + ...	
rigid (Euclidean)	$\begin{bmatrix} R & t \end{bmatrix}_{2 \times 3}$	3	lengths + ...	
similarity	$\begin{bmatrix} sR & t \end{bmatrix}_{2 \times 3}$	4	angles + ...	
affine	$\begin{bmatrix} A \end{bmatrix}_{2 \times 3}$	6	parallelism + ...	
projective	$\begin{bmatrix} \tilde{H} \end{bmatrix}_{3 \times 3}$	8	straight lines	

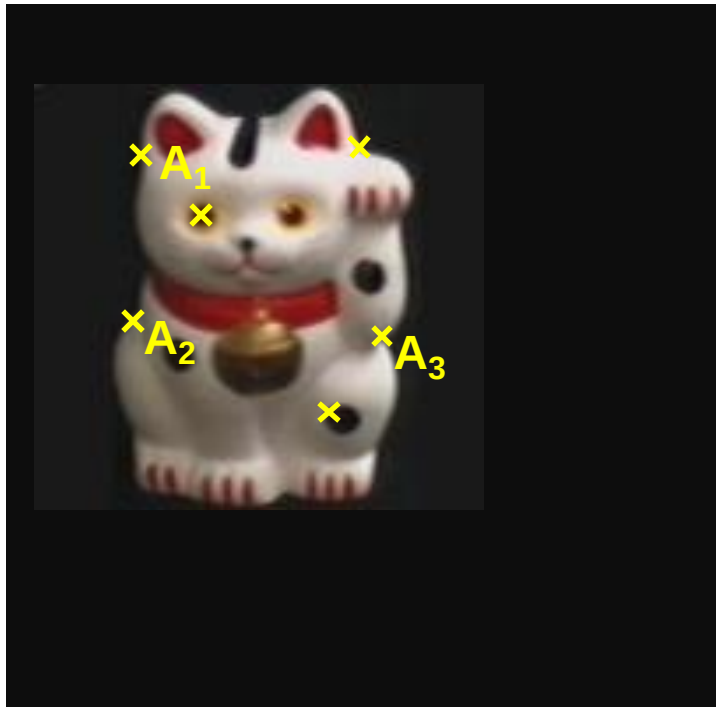
Example: solving for translation



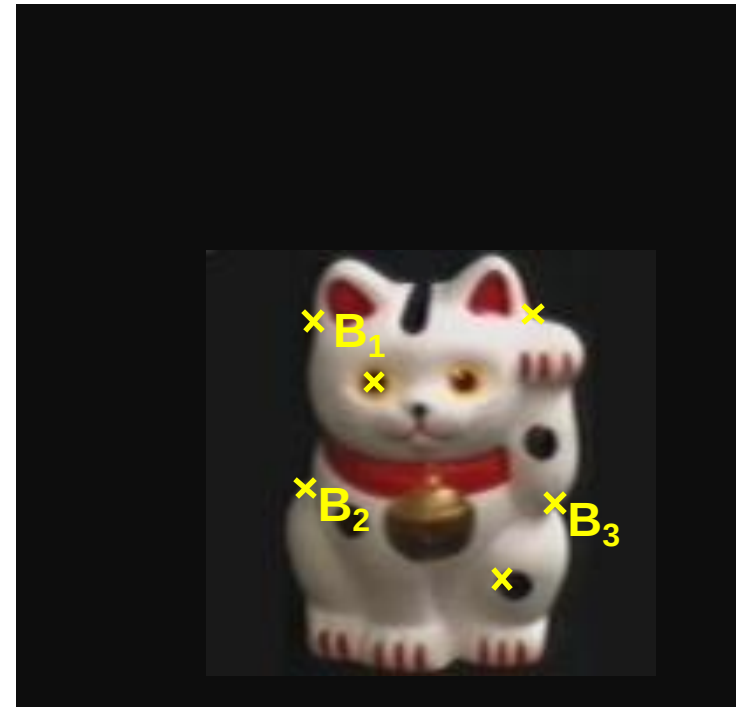
Given matched points in $\{A\}$ and $\{B\}$, estimate the translation of the object

$$\begin{bmatrix} x_i^B \\ y_i^B \end{bmatrix} = \begin{bmatrix} x_i^A \\ y_i^A \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

Example: solving for translation



(t_x, t_y)



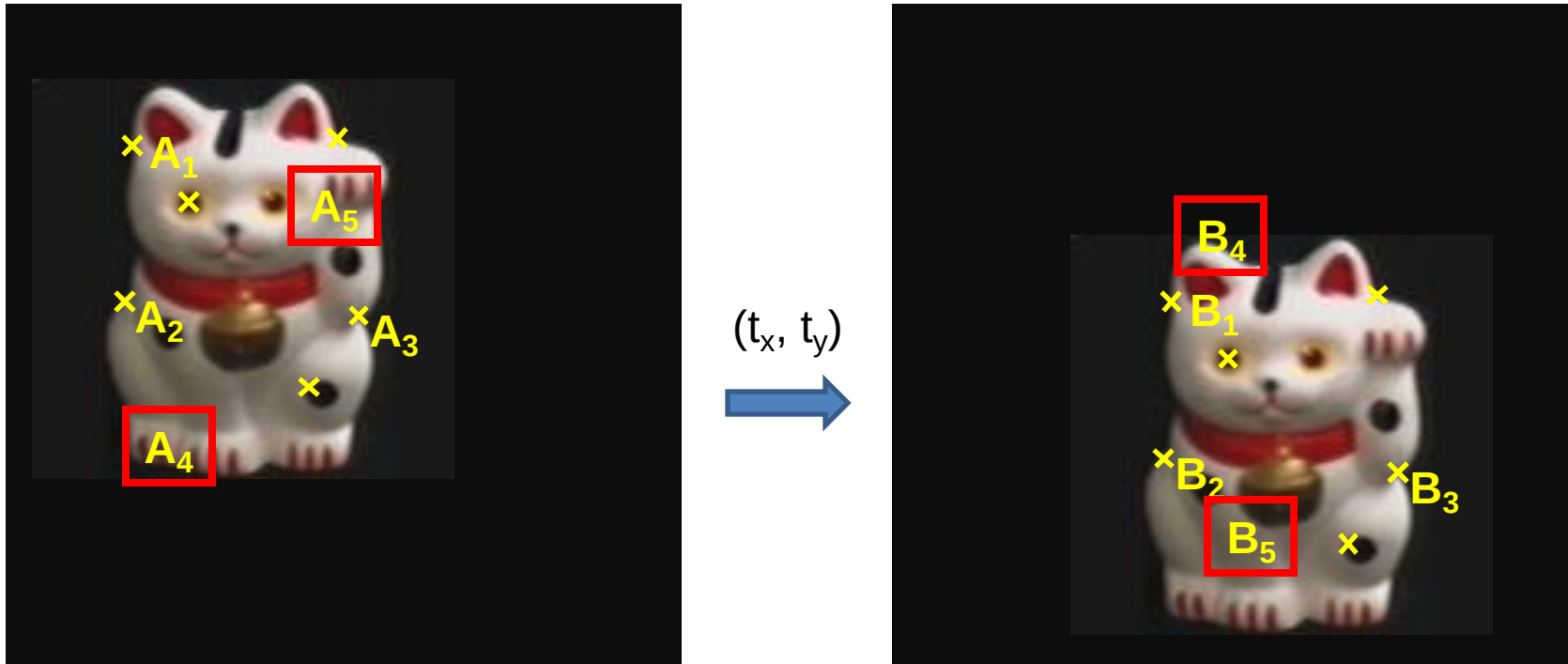
Least squares solution

1. Write down objective function
2. Derived solution
 - a) Compute derivative
 - b) Compute solution
3. Computational solution
 - a) Write in form $Ax=b$
 - b) Solve using pseudo-inverse or eigenvalue decomposition

$$\begin{bmatrix} x_i^B \\ y_i^B \end{bmatrix} = \begin{bmatrix} x_i^A \\ y_i^A \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ \vdots & \vdots \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} t_x \\ t_y \end{bmatrix} = \begin{bmatrix} x_1^B - x_1^A \\ y_1^B - y_1^A \\ \vdots \\ x_n^B - x_n^A \\ y_n^B - y_n^A \end{bmatrix}$$

Example: solving for translation



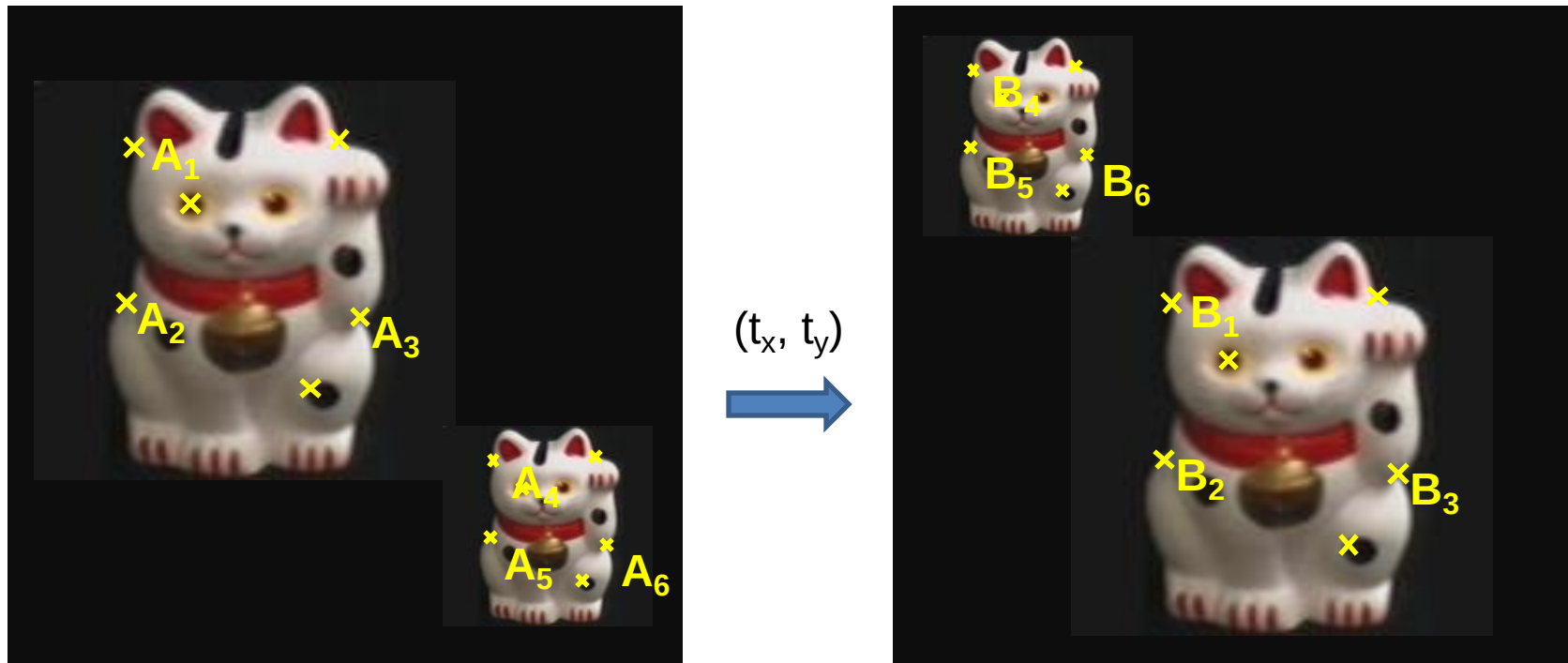
Problem: outliers

RANSAC solution

1. Sample a set of matching points (1 pair)
2. Solve for transformation parameters
3. Score parameters with number of inliers
4. Repeat steps 1-3 N times

$$\begin{bmatrix} x_i^B \\ y_i^B \end{bmatrix} = \begin{bmatrix} x_i^A \\ y_i^A \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

Example: solving for translation



Problem: outliers, multiple objects, and/or many-to-one matches

Hough transform solution

1. Initialize a grid of parameter values
2. Each matched pair casts a vote for consistent values
3. Find the parameters with the most votes
4. Solve using least squares with **inliers** (points given vote to the winning bucket)

$$\begin{bmatrix} x_i^B \\ y_i^B \end{bmatrix} = \begin{bmatrix} x_i^A \\ y_i^A \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$